

Project / Dissertation Statement of Impact [Dubai Campus Disruption 2026]



As a result of the conflict in the Middle East, teaching and learning activities for students at Heriot-Watt University Dubai campus were disrupted. The time available for completion of Final Year Projects / Dissertations was curtailed. In addition, your work may have been impacted in other ways.

The university is committed to ensuring no student is academically disadvantaged by the conflict. You are invited to outline here how the disruption impacted the development of your project / dissertation. This statement will provide context for markers so that your work can be assessed fairly, with due appreciation of context.

Student Name:	Yash V Nadkarni
Student ID Number:	H00410472
Programme:	BSc (Hons) Computer Science (Cyber Security)
Dissertation / Project Title:	A Hybrid Deep Learning-Based Framework for Phishing Website Detection
Supervisor:	Dr. MD. Azher Uddin

Statement of Impact of the Disruption on Your Project / Dissertation

1. What specific impact did the disruption due to the conflict and Dubai Campus closure have on your dissertation / project? Outline the specific disruption and the sections of your work impacted.

For example:

- I was unable to access the labs, so the experiments outlined in Chapter X could not be completed.
- I was only able to complete 10 of a planned 25 interviews, so my findings and analysis are based on a smaller sample than intended.
- I was unable to complete all the planned reading due to disrupted work for X days.

Please be factual and concise. (maximum 100 words)

For my experimentations, I needed multiple computers to run different configurations of my framework to identify the most optimal one, so that I could save time and be able to access my main PC for my other courses. Before this conflict, I used the university lab computers to carry out those experiments. Due to being unable to access the labs now, I could not carry out those experiments and so there's a high chance my final framework implementation isn't the most optimal one.

Project / Dissertation Statement of Impact [Dubai Campus Disruption 2026]

2. What adjustments have you made to your Project / Dissertation as a result of this curtailment?

For example:

- I focused on extending the literature review and developing a detailed theoretical framework.
- I provided an extended description of the intended methodology and experimental design.
- I analysed the partial data I had collected before the disruption.
- I adapted data collection from face-to-face interviews to online.
- I curtailed my literature review and focused on xxx aspects of the topic.

Please be factual and concise. (*maximum 100 words*)

To make up for being unable to access the labs, I did research on more ways to optimize the model. Instead I found a pre-processing method called Random Over Sampling that helped boost my results. I have also extended my literature review and comparison of final works to include more studies pertaining to the two chosen datasets.

Student Signature (*type your name*): Yash V Nadkarni

Date: 01/04/2026

Copy this page and insert it into your Dissertation / Project Report in front of your title page.

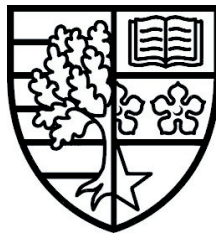
A Hybrid Deep Learning-Based Framework for Phishing Website Detection

by Yash V. NADKARNI

Honours Dissertation

Submitted in partial fulfilment for the degree of
BSc (Hons) Computer Science (Cyber Security)

Supervised by Dr. MD. Azher UDDIN



HERIOT-WATT UNIVERSITY
School of Mathematical and Computer Sciences

1st April 2026

The copyright in this dissertation is owned by the author. Any quotation from the dissertation or use of any of the information contained in it must acknowledge it as the source of the quotation or information.

AUTHORSHIP DECLARATION

Course code and name	F20PA - Research Methods and Requirements Engineering
Type of assessment	Individual
Coursework title	A Hybrid Deep Learning-Based Framework for Phishing Website Detection
Student name	Yash V. NADKARNI
Student ID	H00410472

By signing this form:

- I declare that the work I have submitted for individual assessment or the work I have contributed to a group assessment is entirely my own. I have not taken the ideas, writings or inventions of another person and used them as if they were my own. My submission or my contribution to a group submission is expressed in my own words. Any uses made within this work of the ideas, writings or inventions of others, or of any existing sources of information (books, journals, websites, etc.) are properly acknowledged and listed in the references and/or acknowledgements sections.
- I confirm that I have read, understood and followed the University's Regulations on academic misconduct as published on the [University's website](#), and that I am aware of the penalties that I will face should I not adhere to the University Regulations.
- I confirm that I have read, understood and avoided the different types of plagiarism explained in the University guidance on [Academic Integrity and Plagiarism](#).

Student signature: Yash V. NADKARNI

Date: 1st April 2026

Your work will not be marked if a signed copy of this form is not included with your submission.

ACKNOWLEDGEMENTS

I would first like to give my deepest thanks to my supervisor, Dr. MD. Azher Uddin for his constant support and guidance throughout the development of this project, without which this would not have been possible.

I would like to extend my thanks to my close friends and family for providing me with their love and support through the hardest of times ensuring that I give this project my best.

Lastly, I would like to give my thanks to the rest of the Heriot-Watt University professors and faculty for providing me with the skills and foundation to pursue this project.

ABSTRACT

The rampant growth of online-based services has led to an escalated dependence on the internet for daily communication, finance and more. This dependence has significantly increased the prevalence of phishing attacks that exploit human trust and technical vulnerabilities to steal sensitive personal information. Modern phishing techniques have become more sophisticated and complex, resulting in traditional detection systems struggling to adapt to this more dynamic and adaptive evolution, often failing to generalize past known patterns. Existing studies do not take into account and incorporate sequential temporal dependencies within their phishing detection systems, limiting their ability to capture complex relationships between features. To address this prevailing limitation, a novel hybrid deep learning framework proposed by this dissertation aims to enhance the accuracy and efficiency of detecting phishing websites by employing a 1-Dimensional Convolutional Neural Network (1D-CNN), integrated with a Bidirectional Gated Recurrent Unit (BiGRU). Together, the framework is able to obtain higher-level patterns from URLs, along with any bidirectional temporal dependencies between them, thus allowing the model to capture complex patterns and correlations from the URLs. The framework was evaluated using two widely used datasets, namely the UCI Phishing Websites dataset and the Mendeley 2020 phishing dataset, ensuring exposure to diverse feature distributions for added consistency. Experimental results demonstrate that the proposed hybrid model achieves a peak accuracy of 97.92%, outperforming several traditional and baseline approaches while maintaining strong generalization capability. As such, the proposed hybrid framework seeks to contribute to the development of more adaptive and durable phishing detection systems for the protection of user integrity in this every evolving cyberspace.

Keywords: Phishing Detection, Phishing Websites, URLs, Hybrid Framework, Deep Learning, 1-Dimensional Convolutional Neural Network (1D-CNN), Bidirectional Gated Recurrent Unit (BiGRU), Framework Evaluation.

TABLE OF CONTENTS

Authorship Declaration	i
Acknowledgements	ii
Abstract	iii
Table of Contents	iv
1 Introduction	1
1.1 Background and Motivation	1
1.2 Aim and Objectives	2
1.3 Organisation	2
2 Background Research	3
2.1 List-Based Systems	3
2.2 Heuristics-Based Systems	3
2.3 Visual Similarity-Based Systems	4
2.4 Machine Learning-Based Systems.	4
2.5 Critical Analysis of Related Work.	7
2.6 Comparison of Related Work	8
3 Methodology	9
3.1 Data Pre-Processing	9
3.2 Convolutional Neural Network.	10
3.3 Bidirectional Gated Recurrent Unit	12
3.4 Classification	13
4 Requirements Engineering	15
4.1 Functional Requirements	15
4.2 Non-Functional Requirements	16
4.3 Hardware & Software Requirements	16
5 Evaluation	17
5.1 Datasets	17
5.2 Evaluation Metrics.	17
5.3 Implementation Setup	19
5.4 Results Analysis.	20
6 Conclusions	29
6.1 Limitations.	29
6.2 Scope for Future Work	30
References	31
A Project Management	34
A.1 Project Scope & Deliverables.	34
A.2 Project Plan & Gantt Chart	35
A.3 Risk Analysis	41

B	Professional, Legal, Ethical and Social Issues	42
B.1	Professional Issues	42
B.2	Legal Issues	42
B.3	Ethical Issues	42
B.4	Social Issues	42

1 Introduction

In the modern digital age, day-to-day life has become completely intertwined with the internet, which now serves as a global hub for all things pertaining to communication, finance, information and much more [3]. Most individuals and organizations now rely heavily on the internet for their daily social or productive activities such as social media, banking, education, entertainment, healthcare, etc. However, this evolution in technology has also seen an evolution in cyber threats from malicious actors who seek to exploit online vulnerabilities to compromise privacy and integrity. They utilize several techniques of cybercrime to carry this out, thus disrupting the daily flow of life online [31].

1.1 Background and Motivation

Phishing is a common term used in the digital age that is loosely derived from the process of ‘luring’ in the fishing activity. [12] describes it as the practice of deceiving users by masquerading malicious sources as legitimate. These sources, through which such a scam is usually carried out include websites, emails, SMS, etc. The attacker masks these websites and emails to impersonate legitimate companies such as government portals, medical sites, banks and finance websites and more. The main purpose of this cyberattack is to extract personal sensitive information from victims like passwords which are then used by the attacker for their own benefit [31]. In certain cases, these phishing attacks deceive victims into unknowingly installing malware and viruses onto their computers making them vulnerable to other forms of cyber-attacks.

Over the years, different forms of phishing attacks have been developed by hackers [3]. The common denominator in these attacks is the luring of victims to access a phishing site. Spoofing Emails are malicious emails sent by attackers that are written similarly to their legitimate counterparts with forged sender’s address. Spear-Phishing is a form of Spoofing that targets specific individuals and organizations. Whaling is its subset that involves high ranking targets for obtaining classified information. SMiShing is an attack that uses SMS mobile services instead of emails while Vishing sees the attacker, pretending to be a high-ranking individual, scams the victim through a phone call.

According to [31], the main aspect of developing a phishing website, email, etc. is to make it resemble the official counterpart as close as possible using official logos and layouts so that the victim does not question its authenticity. [44] shows that there is one surefire way to tell whether a website is legitimate or not, and that is by its URL. A Uniform Resource Locator (URL) is the unique defining address of a specific resource available on the internet. Any malicious website seeking to impersonate its legitimate counterpart will have an irregular URL that will give away its legitimacy when examined by experienced internet users. An analysis shows that these URLs would include unnecessary special characters (e.g., @, /, -), or misspelled domain names (e.g., gooogole, fac3bo0k).

Unfortunately, due to the fast-moving lives of common folk, most people tend to not notice the irregular URLs. They’re more focused on completing their everyday tasks, which makes them extremely vulnerable to phishing attacks. [32] showcases that during the Covid-19 pandemic, there was a huge surge in phishing attacks with its frequency increasing

by 4 times during this period. [40] tells us that these phishing attacks used covid related domains to lure people by allegedly providing methods to stop the outbreak. According to the Anti-Phishing Working Group (APWG), their Phishing Activity Trends Report for 2025 [14] reveals that over 2 million phishing attacks have occurred in the first half of 2025 with the financial sector being frequently targeted.

As seen in Section 2, the integration of temporal dependencies modeling combined with extraction of higher-level feature patterns in phishing detection remains relatively unexplored. Despite CNN's ability to capture structural patterns of URLs, it is limited in learning temporal features. The proposed 1D-CNN-BiGRU hybrid framework addresses this limitation by being capable of capturing forward and backward temporal dependencies of the lexical features, thus allowing for detecting deeper relations and patterns of the URL's structure.

1.2 Aim and Objectives

Aim: To design, implement and evaluate a hybrid deep learning framework for the purpose of detecting phishing websites and assessing whether the hybrid model outperforms traditional ML based systems in terms of accuracy on standard phishing datasets.

Objectives:

- (1) Acquire and preprocess publicly available phishing datasets.
- (2) Develop a framework for a hybrid deep learning model.
- (3) Utilize a 1-Dimensional Convolutional Neural Network (1D-CNN) for extracting lexical features and patterns.
- (4) Utilize a Bidirectional Gated Recurrent Unit (BiGRU) to extract the temporal features and sequential dependencies.
- (5) Train the hybrid framework on the preprocessed datasets.
- (6) Evaluate the performance of the framework by means of accuracy, precision, recall and f1-score.
- (7) Compare the evaluated performance of proposed framework against existing ML-based phishing detection models.

1.3 Organisation

The remainder of this dissertation is structured as follows. Section 2 presents the background research and analysis of prior studies. Section 3 describes the proposed hybrid framework, while Section 4 outlines the necessary requirements analysis. Section 5 evaluates the framework, including datasets, metrics, and ablation study results. Section 6 concludes the dissertation. Finally, the back matter includes appendices covering the Gantt chart, risk analysis, Professional, Legal, Ethical and Social (PLES) considerations, and references.

2 Background Research

While increasing user's knowledge on phishing attacks is imperative, it is evident that it is not enough. The development of automated software capable of detecting and warning users of potential phishing attacks has seen significant growth overtime. This section shall cover the solutions proposed by previous studies conducted for detecting phishing sites. They are categorized based on the methodologies used.

2.1 List-Based Systems

Being the earliest form of anti-phishing protection, the core concept of list-based systems is the classification and storage of websites into white and black lists based on their legitimacy. Comparatively, these are simpler due to their fast lookups and require less system costs.

2.1.1 Whitelists. A whitelist is a list or database containing only those websites that are confirmed to be legitimate. When the user enters a site, the system cross-references the site's URL with the whitelist to see if the website is safe. If a website is not found, it is flagged as skeptical.

Earlier works include a system created by [10] that stores legitimate websites based off their IP addresses. Though the system provides a warning for unknown sites, it fails to classify new legitimate websites, showing the need to be constantly updated.

A popular system developed by [16] in 2016 that achieved an accuracy rate of 86.07%, includes the auto-updating of the whitelist on the client side to factor in newer legitimate websites. Their methodology included matching the IP address and extracting features from the website's source code.

2.1.2 Blacklists. Opposite a whitelist, a blacklist contains only the websites that are known to be malicious. Similar to how whitelists behave, a blacklist-based system cross-references the stored websites' features (e.g., URL, IP address) with those of the visited website. If a match is found, website access to the user is denied.

A zero-day phishing attack is a form of zero-day exploit [15] that involves a new phishing attack from a malicious website that has recently been created and is not yet known by blacklist systems. The period from the website's conception to its addition to a blacklist is called the Window of Vulnerability [11]. This type of attack is always resistant to blacklists no matter how frequently they are updated, thus highlighting the need for more adaptive systems. PhishTank [26] and Google Safe Browsing [9] are some of the most popular phishing black list services available.

2.2 Heuristics-Based Systems

The insufficiency of list-based systems made it evident that more dynamic solutions were needed. Heuristics-based systems [11], aim to extract a website's properties to determine its legitimacy. This can range from lexical URL attributes (e.g., @, /, -) to the HTML source code of the webpage.

[43] developed CANTINA which analyzed the webpage's text to extract keywords and terms. Using TF-IDF, the keywords were searched up in search engines for assessment.

If the website appeared in the results, it was deemed legitimate, else malicious. Despite exhibiting a high true positive rate, it suffers from being restricted to the English language due to limitations of TF-IDF. CANTINA+, a more enhanced system developed by [41] employs 8 more features based off visual similarities, HTML DOM and more. PhishNet from [27] which outperformed Google Safe Browsing [9], also uses heuristics through URL analysis to predict newer phishing sites.

Heuristics based systems laid the foundation for the transition towards Machine Learning based approaches that emphasize more on feature-based detections.

2.3 Visual Similarity-Based Systems

Another common approach to detect phishing websites is to analyze their screen shots to form predictions. These systems extract features from these webpages that alter the visual layouts and appearances. The system proposed by [17] extracts the position of elements, color schemes, CSS and HTML DOM trees, pixel features, etc. while another [39] employs a dual scale framework that analyzes global (layout) and local similarity (logos).

This methodology addresses zero-day attacks; however, it requires significant computational power for the processing of images. Furthermore, aesthetically different layouts due to bad CSS codes can result in a large percentage of false negatives [11].

2.4 Machine Learning-Based Systems

To further combat the evolving techniques used by phishing attacks, researchers have turned to ML based techniques to develop adaptive detection systems driven by data.

While heuristics systems rely on manually defined algorithms such as TF-IDF in the case of CANTINA [43], ML systems identify patterns and relations from data to form distinctions. This shift towards ML based systems marked a massive advancement that gave rise to superior predictive systems that are able to handle zero-day attacks.

ML models need appropriate datasets for proper training. As described by [40], feature selection a.k.a feature engineering is the method of selecting specific types of data that will aid the model in identifying patterns. For example, the number of '/' symbols in a URL is a feature. It is crucial to select only the right set of features for training the model, as using pointless or noisy features will result in higher computational costs and time which could harm the model's accuracy.

As the sites are classified as legitimate or phishing/malicious, these ML models are considered a binary classification problem. Over time, several different ML models have been developed with each one having different performance rates when it comes to the detection of phishing websites.

In 2013, one of the earliest studies involving ML based techniques for phishing detection saw [18] evaluating several different algorithms to test out their effectiveness. The dataset consisted of 37,000 phishing & legitimate URLs obtained from public whitelists and blacklists such as PhishTank [26]. After performing lexical analysis on, they then used these lexical features to train 4 different ML based models: Naïve Bayes, J48 Decision Tree, k-NN and SVM. Evaluation saw J48 Decision Tree outperforming the rest with a 93% accuracy.

In 2018, [36] performed a similar study where they evaluated the performance of Decision Trees, RF and GBM using various features extracted from URLs from PhishTank [26]. A technique called Principal Component Analysis (PCA) was applied to combine features to create new features. Upon evaluating their results, they found that the RF model achieved the highest accuracy rate of 98.40% on the PCA applied dataset. Another study from [29] applied the Classification and Regression Tree (CART) algorithm on the UCI_2015 [24] phishing dataset which classifies based on predictor values to reach a target value, achieving an accuracy of 91%.

One study [30] utilized the eXtreme Gradient Boosting (XGBoost) algorithm on the UCI_2015 [24] phishing dataset and achieved an accuracy of 97.45%, demonstrating the strong performance of boosting-based ensemble methods. Similarly, another study from [28] utilized an XGBoost model with GridSearchCV tuning on the Mendeley_2020 dataset [38], reporting an accuracy of 96.34%, demonstrating the strength of optimizing gradient boosting techniques.

A study from [20] proposed a multilayer stacked ensemble model which combines the predictive strengths of several ensemble ML algorithms to detect phishing sites. Ensemble & deep learning models such as MLP, kNN, RF, Logistic Regression and XGB were employed across 3 layers, with the outputs of one layer serving as inputs to the next. After evaluation, the model achieved a high accuracy (97.76%, 98.90%, 96.79%) across 3 different datasets. However, the employment of multiple ML algorithms simultaneously across several layers, results in incredibly high computational costs. Later studies indicate that the utilization of a single algorithm with proper feature selection is sufficient in achieving high accuracy.

A study by [37] employed Ensemble Learning through utilizing multiple learning models through a voting mechanism to output a final classification based on the UCI_2015 dataset [24]. The proposed study achieved a competitive accuracy of 95.40%. Similarly, another study [42] proposed an ensemble learning-based model (EnLeM) with a hard voting mechanism and mutual information-based feature selection to achieve an accuracy of 97.21% on the UCI_2015 dataset [24], improving on the previous ensemble voting study.

The system proposed by [6] highlights the superiority of RF with an accuracy of 97.3% on a more popular UCI dataset [24], while another from [4] using a semantic features URL dataset [34] saw RF outperforming 15 other ML algorithms to achieve a 99.06% accuracy.

An experimental study by [1] examined the impact of dataset balancing, hyperparameter tuning and feature selection on the performance of phishing detection. Utilizing the UCI [24] and Mendeley 2018 [34] datasets, each model was trained and tested twice, once with and without optimization. For the UCI, fine-tuning improved accuracy, while it remained the same for Mendeley 2018. From this, it can be inferred that proper optimization can maintain or even improve accuracy while considerably reducing computation cost.

A study from [2] proposes an optimized stacking ensemble model for phishing detection using the UCI [24] Mendeley 2018 [34] and Mendeley 2020 [38] datasets. At first, RF & LightGBM had achieved the best accuracies, however, after applying Genetic Algorithm (GA)-based ensemble classifiers as seen in Fig.1, there was a shift in performance rankings. GA-based Gradient Boosting achieved the highest accuracy on UCI (97.13%), while GA-based XGBoost did the same for Mendeley 2018 and 2020 (98.57% & 97.35%). These results

emphasize that despite RF performing strongly, feature selection and algorithm tuning can yield superior results.

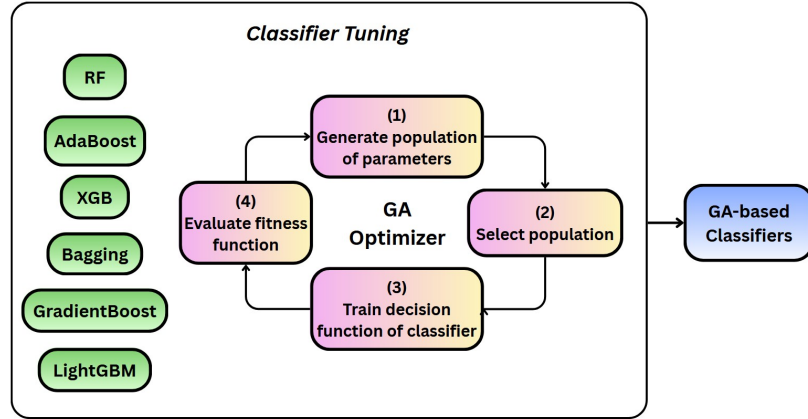


Fig. 1. Application of GA to ML classifiers by [2]

One study [32] saw the proposal of a Rough Set Theory-Based Hybrid Feature Selection (RSTHFS) method to reduce the feature dependency in ML models. By combining Rough Set Theory (RST), Correlation-Based Feature Selection (CFS) and Cumulative Distribution Function (CDF), a feature cut-off is identified which reduced the feature count by 69.11% on 3 datasets [24], [38] and [34]. Evaluating the performance of the ML models, it was evident that RSTHFS was able to achieve similar accuracy while reducing runtime by 30%, highlighting the value of effective feature selection.

The diverse and more sophisticated nature of URLs from phishing attacks over the last couple years have compelled researchers to turn to Deep Learning (DL)-based systems. In contrast to ML models, DL models are able to extract more complex patterns from data that would otherwise require extensive human defined features. This is especially powerful in the defense against zero-day phishing attacks.

The study from [5] evaluated and compared traditional ML models against a DL model called Convolutional Neural Network (CNN) that extracts spatial patterns. The CNN trained on 20 epochs utilizing 3 dense layers with ReLU and sigmoid activation functions. Evaluation on 2 datasets [24], [38], saw it outperforming ML models with an accuracy of 99%. A Feedforward Neural Network architecture proposed by [25] achieved an accuracy of 95.53% when trained and tested on the Mendeley_2020 dataset [38], indicating that simple deep learning architectures can attain strong performance.

Another study conducted by [7] analyzed and compared the performance of 3 DL models: CNN, LSTM and a LSTM-CNN hybrid system on the ISCX-URL2016 [13] dataset with CNN outperforming both LSTM and LSTM-CNN hybrid models by a difference of 3%, obtaining an accuracy of 99.2%. One study from [35] found that employing a Gated Recurrent Unit (GRU), like in Fig. 2, to a Recurrent Neural Network (RNN) increased the accuracy from 74% to 99.18%.

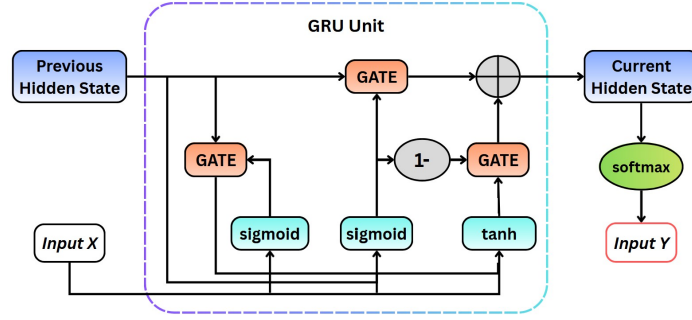


Fig. 2. Illustration of the architecture of a GRU Unit used by [35]

2.5 Critical Analysis of Related Work

Research on phishing detection has produced several methodologies, yet each exhibits its own limitation. List-based systems [10] demonstrate simple implementations but require constant updating making them ineffective against zero-day attacks. This highlights the need for adaptive methods over purely reactive systems. Systems analyzing visual features [17] resolve this, but require complete webpage rendering, resulting in high computational costs in addition to a high rate of false negatives from bad designs [11]. Another attempt comes from heuristics systems [43], [41] and [27] using predefined algorithms and fixed heuristics. However, these suffer from hand-crafted logic that attackers can easily bypass by altering URL keywords and structure, making them lack adaptability to evolving strategies.

Adaptability through learning solves this limitation. Machine learning-based systems demonstrate a significant improvement over heuristics by learning statistical patterns from data instead of relying on predefined rules with ensemble methods such as Random Forest [36] [6] [4] consistently outperforming all other classifiers. However, their dependence on extensive manual feature engineering requires that features be repeatedly redesigned as phishing tactics evolve. In addition, features that are effective on one dataset may fail on another, highlighting the need for an even more adaptive approach that also performs consistently across different data. Complex ML systems [20] [2] show similar performances but incite high computational costs without resolving hand-crafted feature dependence. In that regard, deep learning systems prove to be more efficient than ML systems as they eliminate the need for manual feature engineering by learning representations and patterns directly from raw data. CNN models [5] [7] capture local spatial patterns while GRU models [35] extract temporal dependencies to improve accuracy. However, most deep learning models treat URL data as purely spatial or purely sequential, with each model struggling to capture information from the other type.

CNNs struggle to capture sequential relationships between paths, subdomains and queries while GRUs struggle to extract spatial cues such as suspicious characters. This highlights the lack of joint architecture combining both dependencies in phishing detection which this dissertation aims to address by proposing a hybrid framework utilizing a 1D-CNN and BiGRU offering improved adaptability with reduced reliance on manually-crafted features.

2.6 Comparison of Related Work

Study	Dataset	Model	Accuracy
[18]	PhishTank [26]	J48 Decision Tree	93%
[29]	UCI_2015 [24]	CART	91%
[30]	UCI_2015 [24]	XGBoost	97.45%
[28]	Mendeley_2020 [38]	XGBoost	96.34%
[20]	UCI_2015 [24]	MLSELM	97.76%
[20]	Mendeley_2020 [38]	MLSELM	96.79%
[37]	UCI_2015 [24]	Ensemble Learning	95.40%
[42]	UCI_2015 [24]	EnLeM	97.21%
[4]	Mendeley_2018 [34]	Random Forest	99.06%
[2]	UCI_2015 [24]	GA-based Gradient Boost	97.13%
[2]	Mendeley_2020 [38]	GA-based XGBoost	97.35%
[7]	ISCX-URL2016 [13]	CNN	99.20%
[25]	Mendeley_2020 [38]	FNN	95.53%
[35]	Kaggle Dataset, PhishTank [26]	RNN-GRU	99.18%

Table 1. Comparison of accuracy from related studies.

3 Methodology

As illustrated in Fig. 3, the proposed phishing detection framework utilizes a hybrid approach employing a 1-Dimensional Convolutional Neural Network (1D-CNN) and a Bidirectional Gated Recurrent Unit (BiGRU) for classifying websites as phishing or legitimate. This 2-system approach is designed to utilize both the 1D-CNN's ability to capture local lexical patterns and the BiGRU's capability to model global sequential dependencies from the URL features, thus providing a novel, robust and effective model tailored for phishing detection.

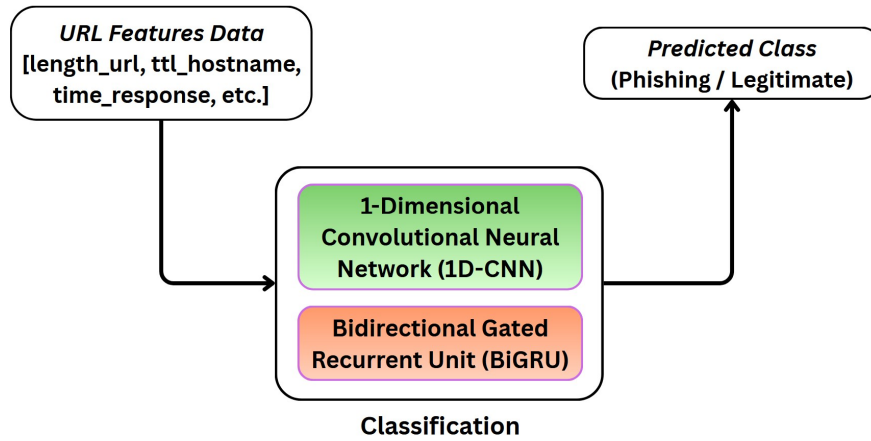


Fig. 3. Architecture of Proposed Framework

3.1 Data Pre-Processing

Raw datasets often contain missing values, inconsistent entries, noisy records, and features with significantly different numerical scales, all of which can negatively affect the performance of machine learning models trained on them. The datasets selected for this project are no exception, making data pre-processing a critical stage in the development of the proposed hybrid framework. Implementations based on neural network architectures are particularly sensitive to the numerical distribution of input features. As such, improperly scaled data, missing values, or noisy observations can lead to reduced classification accuracy, unstable training behavior, and slower model convergence.

To mitigate these issues, preprocessing techniques are applied to transform the raw data into a clean, consistent, and machine-learning-ready format. This process typically involves identifying and handling missing or incomplete values, removing duplicate or irrelevant records, and correcting inconsistencies within the dataset. Additionally, feature scaling and normalization techniques are applied to ensure that all input variables fall within comparable numerical ranges, preventing features with larger magnitudes from disproportionately influencing the learning process.

For safeguarding methodological consistency and preventing data leakage, pre-processing was applied using parameters obtained from training data exclusively to the training set and then subsequently the validation and test sets. This pipeline ensures that the training set does not get influenced by gaining unintended information from the validation and test sets, thus conserving its integrity during evaluation.

3.2 Convolutional Neural Network

The first component of the hybrid framework's processing pipeline begins with a one-dimensional Convolutional Neural Network (1D-CNN), which is employed to capture local structural patterns from the feature vectors extracted from URLs. Convolutional Neural Networks (CNNs) are a category of deep learning models designed to automatically learn hierarchical feature representations from input data through convolutional operations. Traditionally, CNNs have been widely used in computer vision tasks, including image classification, object detection, and pattern recognition, due to their ability to capture spatial hierarchies and local patterns within images [23]. Their architecture typically consists of convolutional layers, activation functions, pooling layers, and fully connected layers that work together to progressively extract higher-level representations from raw input.

Although CNNs were originally developed for two-dimensional image data, their architecture can be adapted to one-dimensional sequential data. In this configuration, a 1D-CNN performs convolution operations along a single dimension, enabling the model to detect local feature patterns within sequences such as time-series data, text, or structured feature vectors [22]. In the context of phishing URL detection, 1D-CNNs have proven effective due to their ability to efficiently capture local dependencies and discriminative patterns within textual or feature-based representations of URLs. Previous studies have demonstrated the effectiveness of convolutional architectures in text classification tasks by identifying key n-gram level patterns and structural features within input sequences [21]. By applying convolutional filters across the feature vectors, the model can automatically learn patterns that distinguish phishing URLs from legitimate ones.

The CNN component in the proposed hybrid framework therefore functions as a feature extractor, identifying and learning important local dependencies within the input feature vectors before passing them onto the BiGRU component for temporal extraction. Fig.4 displays the visual representation of the architectural structure of a 1D-CNN employed within the proposed hybrid framework.

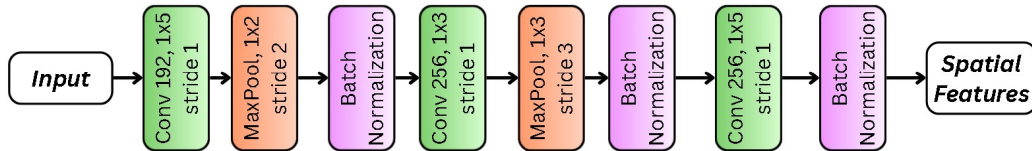


Fig. 4. Illustration of employed 1D-CNN model.

3.2.1 *One-Dimensional Convolutional Layer.* :

The convolutional layer forms the core component of the 1D-CNN architecture and is responsible for extracting spatial feature patterns from the input feature vectors. In this layer, a set of learnable filters performs convolution operations across small regions of the input, enabling the model to detect meaningful spatial structures. As the filters slide across the input sequence, they generate feature maps that highlight the presence of specific patterns learned during training. Several key parameters control the behavior of the convolutional layer. The number of filters determines how many distinct feature patterns can be learned simultaneously, with each filter capturing a different abstraction of the input data. The kernel size defines the size of the local region examined during convolution, where larger kernels capture broader contextual patterns while smaller kernels focus on more fine-grained local ones. After each convolution operation, a ReLU activation function is applied to introduce non-linearity, allowing the network to learn complex decision boundaries for phishing detection. Padding is set to 'same' to ensure that the output feature map maintains the same dimensionality as the input, allowing edge features to be preserved. Additionally, L2 kernel regularization is applied to penalize large weights and reduce the risk of overfitting, thereby improving the model's generalization capability. In the proposed architecture, three convolutional blocks are implemented sequentially with 192 filters and a kernel size of 5 in the first layer, followed by 256 filters with a kernel size of 3 in the second layer, and 256 filters with a kernel size of 5 in the third layer, all using ReLU activation and an L2 regularization factor of 2×10^{-5} . This progressive configuration enables the network to first capture broader structural patterns and then refine them through deeper layers, ultimately transforming the input feature vector into a hierarchy of informative feature maps that are passed to subsequent layers of the hybrid framework.

Following each convolutional operation, batch normalization is applied to stabilize the training through normalizing feature maps by regulating their mean and variance. This ensures that the features extracted by the convolutional layer remain stable before being passed onto the subsequent layers.

3.2.2 *One-Dimensional Max Pooling Layer.* :

Max pooling plays the role of down sampling the feature maps by selecting the maximum value from a defined window. This is done in order to reduce the dimensionality of the feature maps. A larger pooling size causes greater dimensionality reduction while a smaller pooling size preserves more detail. Max pooling emphasizes the most dominant feature from every local region which in the case of this project's problem space, highlights the strong indicators of phishing while minimizing weaker ones. Additionally, max pooling reduces complexity and controls overfitting.

3.3 Bidirectional Gated Recurrent Unit

Recurrent Neural Networks (RNNs) are a category of neural network models designed to model sequential data by maintaining hidden states that capture dependencies between elements in a sequence, allowing information from previous time steps to influence future predictions. However, traditional RNNs suffer from the vanishing and exploding gradient problems, where gradients become extremely small or excessively large during backpropagation through time. This issue limits the network's ability to learn long-term dependencies within sequences [8]. To address these limitations, gated recurrent architectures such as the Gated Recurrent Unit (GRU) were introduced which incorporate gating mechanisms that regulate the flow of information through the network, enabling the model to retain relevant contextual information over longer sequences while discarding irrelevant patterns [35]. As illustrated in Fig.5, the mechanism consists of two primary gates: the update gate that controls how much information from the previous hidden state is maintained and the reset gate which determines how much information is ignored. These gating mechanisms allow GRUs to selectively preserve useful contextual features while filtering out noise, thereby improving learning efficiency and stability.

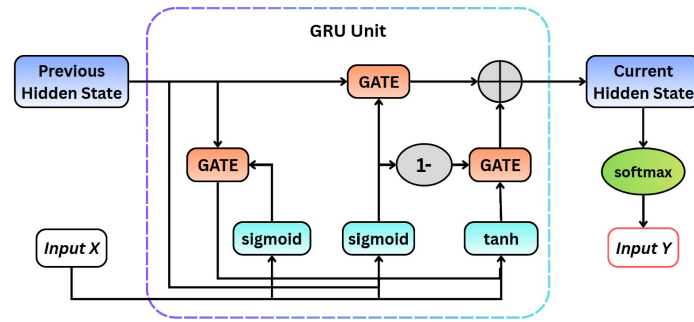


Fig. 5. Illustration of the architecture of a GRU Unit used by [35]

For the proposed hybrid framework, a Bidirectional Gated Recurrent Unit (BiGRU) is utilized to further enhance the model's ability to capture contextual relationships within the feature representations generated by the 1D-CNN. In a bidirectional architecture, two GRU layers operate simultaneously: one processes the input sequence in the forward direction, while the other processes it in the reverse direction which according to [33] can be seen as capturing both past and future feature relationships when making classification decisions. The hidden states generated from both directions are then combined to form a unified representation that incorporates information from both preceding and succeeding elements within the sequence [19]. Fig.6 shows a simplified illustration of the BiGRU layer employed in the proposed framework.

For the purpose of tuning, certain parameters such as the number of GRU units and Dropout Regularization are crucial to obtaining the most optimal model. The number of GRU units determines the dimensionality of the hidden state with each GRU unit contributing to learning complex temporal relationships. Similar to CNN, dropout is applied to improve

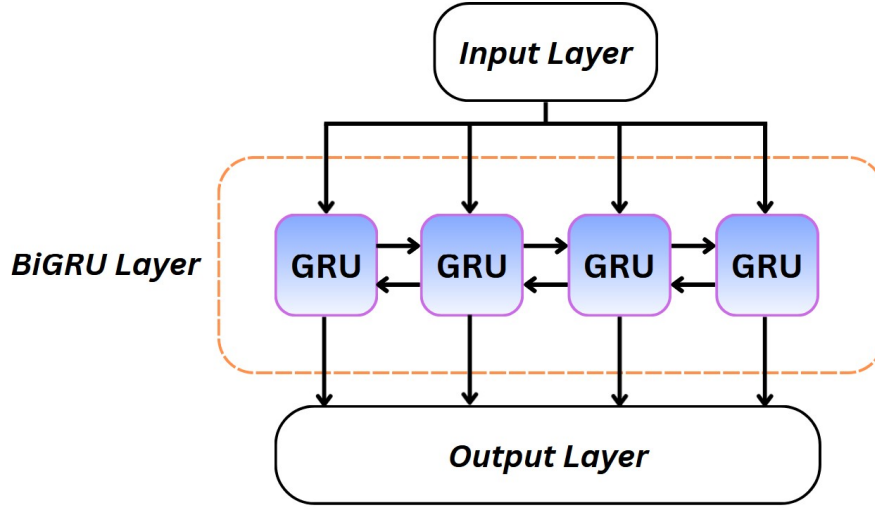


Fig. 6. Illustration of one employed BiGRU layer.

generalization and reduce overfitting by deactivating a random subset of units during training. Lastly the complete sequence of hidden states for every time step is returned to allow for more detailed temporal information to be preserved so that the most relevant features are identified.

For further refinement, a dual pooling strategy is employed. Dual pooling consists of utilizing Global Average Pooling and Global Max Pooling which when concatenated together, compress the sequence of hidden states generated into feature vectors of fixed length while still preserving necessary information. Global Average Pooling provides a generalized representation of the complete input sequence from the overall distribution while Global Max Pooling selects the most dominant feature detected. Combining both methods ensures the model benefits from both generalized contexts and strong localized features, which helps to improve robustness and further enhance performance.

3.4 Classification

Following spatial feature extraction and temporal modeling, the resulting representation is passed to the Fully Connected Layers, known also as Dense Layers. These act as the final decision-making component that perform high level reasoning to map the obtained representation to the final classification output. Each dense layer performs a linear transformation on the feature representation which is then followed by a non-linear activation function. This enables the framework to refine the representation through learning hierarchical feature interactions for classification.

$$y = f(Wx + b) \quad (1)$$

In the formula [23], the input feature vector (x) is multiplied by the weight matrix (W) using dot product and summed to the bias vector (b). The activation function (f) is then applied to obtain the transformation (y). The role of the activation function is to introduce non-linearity for applying decision boundaries for distinguishing between phishing and legitimate websites. Increasing the number of dense layers can allow the model to learn more complex relationships but that complexity may also cause overfitting, thus highlighting the need for a balanced architecture. In addition, dense layers also enable dimensionality reduction. The hybrid framework also employs batch normalization and dropout after each dense layer similar to the 1D-CNN.

The output layer is the final layer of the framework that produces the classification result. Phishing detection is labelled as a binary classification problem; hence, the output layer will consist of a single unit that transforms the result into an output of either 0 or 1. The sigmoid activation [23] function works well for this classification:

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2)$$

The probability generated by the sigmoid activation function represents how confident the framework is that the given input can be classified as phishing. Values closer to 1 indicate higher likelihood of phishing, while values closer to 0 indicate higher likelihood of legitimate website. To obtain a distinct classification, threshold optimization was utilized where the model learns the ideal threshold (t) during training. The threshold can vary in the range $\{0.4, 0.6\}$.

- Output $\geq t$ is classified as Phishing
- Output $< t$ is classified as Legitimate

4 Requirements Engineering

This section shall cover all the necessary requirements that will have to be undertaken in order to ensure the framework is successfully developed. The requirements shall be prioritized based on the MoSCoW method.

4.1 Functional Requirements

ID	Requirement	Priority	Met
FR.1	Dataset Preprocessing	Must	Yes
FR.1.1	Datasets should be cleaned & normalized to remove redundancy and ensure better accuracy.	Must	Yes
FR.1.2	Datasets should be split into training and testing sets.	Must	Yes
FR.1.2.1	Utilize k-Fold Cross Validation for splitting the datasets.	Should	Yes
FR.1.3	Preprocessed datasets should be saved separately for easier access.	Should	Yes
FR.2	Implementation of Models	Must	Yes
FR.2.1	Training traditional ML models on the training set of our dataset.	Must	Yes
FR.2.2	Implementation and integration of 1D-CNN and BiGRU models, optimized with appropriate hyperparameters, to obtain desired hybrid framework.	Must	Yes
FR.2.3	Train the hybrid framework on the preprocessed train set.	Must	Yes
FR.2.4	Test the trained framework on the test set of the datasets.	Must	Yes
FR.3	Performance Evaluation	Must	Yes
FR.3.1	From the model's performance, generate the confusion matrix.	Must	Yes
FR.3.2	Calculate the empirical evaluation metrics using values derived from the confusion matrix.	Must	Yes
FR.3.3	Generate additional performance graphs for further evaluation.	Could	Yes
FR.4	Comparative Analysis	Must	Yes
FR.4.1	Compare & analyze the performance metrics of ML models against that of proposed hybrid framework.	Must	Yes
FR.4.2	Compare the performance of models across the two datasets.	Must	Yes

FR.4.3	Compare the performance of proposed hybrid framework with previous works.	Must	Yes
FR.5	Miscellaneous	Could	No
FR.5.1	Development of browser extension utilizing the hybrid framework.	Could	No

Table 2. Table of Functional Requirements.

4.2 Non-Functional Requirements

ID	Requirement	Priority
NFR.1	Hybrid framework must obtain a high accuracy $\geq 80\%$.	M
NFR.2	Access to models and datasets shall be restricted to the author and supervisor only.	S
NFR.3	Datasets must be stored and processed locally.	M
NFR.4	Throughput during testing phase should be tolerable.	S

Table 3. Table of Non-Functional Requirements.

4.3 Hardware & Software Requirements

ID	Component	Requirement
HR.1	Processor (CPU)	Intel i5 or AMD Ryzen 5
HR.2	Graphics Card (GPU)	Any NVIDIA GPU with 4+ GB VRAM
HR.3	Memory (RAM)	8 GB
HR.4	Storage	10 GB SSD for fast processing
SR.1	Software	• Python 3.9+ • TensorFlow / Keras 2.12+ • NumPy 1.25+ • Pandas 2.0+ • Scikit-learn 1.3+ • Matplotlib

Table 4. Table of Hardware & Software Requirements.

5 Evaluation

5.1 Datasets

For training and evaluating the proposed hybrid framework, two publicly available benchmark state-of-the-art datasets shall be employed, the UCI Phishing Websites Dataset [24] and the Mendeley 2020 Dataset [38]. Utilizing two datasets ensures that our framework performs consistently through evaluation across URL data of varying size and complexities. Both datasets comprise of previously extracted features from existing phishing and legitimate websites, thus eliminating the need for a feature extraction phase.

5.1.1 UCI 2015 Dataset. :

The University of California Irvine Machine Learning Repository contains the Phishing Websites dataset which is one of the most widely used datasets in the field of phishing detection [24]. The dataset comprises 11,055 websites (4898 phishing, 6157 legitimate), each one distinguished by 30 different handcrafted features describing the website's URL's structural and lexical attributes. The 31st feature labels the website as phishing or legitimate, thus providing a clear binary classification problem.

5.1.2 Mendeley 2020 Dataset. :

With contrast to the UCI dataset, the Mendeley 2020 Dataset [38] comprises of 111 features extracted from 88,647 websites (30,647 phishing, 58,000 legitimate). The features extracted are more comprehensive encompassing the URL's lexical properties (length, entropy), domain attributes (age, DNS) and network attributes (IP, ssl validity). The proposed hybrid framework based on deep learning models will surely benefit from the well-suited size and variety of this dataset.

5.2 Evaluation Metrics

From testing the hybrid framework, 4 metrics can be obtained directly which are then utilized to generate the confusion matrix as illustrated in Table. 5.

- **True Positives (TP):** The number of phishing websites correctly classified by the framework.
- **True Negatives (TN):** The number of legitimate websites correctly classified by the framework.
- **False Positives (FP):** The number of legitimate websites incorrectly classified as phishing by the framework.
- **False Negatives (FN):** The number of phishing websites incorrectly classified as legitimate by the framework.

From the confusion matrix, we can derive several performance metrics that will precisely indicate how effective the framework is and where it stands. These metrics are based on the standard empirical classification metrics used to evaluate performance.

	Phishing	Legitimate
Phishing	TP	FP
Legitimate	FN	TN

Table 5. Confusion Matrix from testing the hybrid framework.

5.2.1 Accuracy. :

Proportion of correctly classified websites among all predictions. It represents the overall performance of the hybrid framework in phishing detection in balanced datasets. However, both datasets that the hybrid framework will utilize are imbalanced with legitimate websites dominating. Thus, additional metrics are needed.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

5.2.2 Precision. :

Proportion of correctly classified phishing websites among all websites classified as phishing by the framework. A high precision indicates that the model produced very few false positives, thus preventing the unnecessary blacklisting of legitimate websites.

$$Precision = \frac{TP}{TP + FP} \quad (4)$$

5.2.3 Recall (Sensitivity). :

Proportion of correctly classified phishing websites among all phishing websites. It can be seen as the accuracy of correctly predicting only phishing websites. A high recall indicates that the framework is able to detect and capture most phishing attacks, thus reducing the chances of letting real threats slip.

$$Recall = \frac{TP}{TP + FN} \quad (5)$$

5.2.4 F1-Score. :

Represents the harmonic mean of the precision and recall. It balances FP and FN and is especially useful for imbalanced datasets.

$$F1 - Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (6)$$

5.3 Implementation Setup

The system used for carrying out the experiment was an ASUS Vivobook with AMD Ryzen 9 5900HX with Radeon Graphics, along with a 32 GB RAM and NVIDIA RTX 3050. Python3 was the programming language employed due to access to its rich deep learning libraries of which tools from Tensorflow, Keras and Scikit-Learn were utilized for designing, training and evaluating model architecture. Additionally, the tools were also utilized for pre-processing and obtaining evaluation metrics. Supporting libraries such as Pandas, Numpy, matplotlib and seaborn were used for dataset manipulation, calculations and visualizations.

This project utilizes two publicly available state-of-the-art datasets that differ in their feature representation and data quality. Thus, they require specific individual pre-processing strategies to ensure consistency and handle invalid values. For the UCI_2015 dataset, the classification labels were converted from the range -1, 1 to 0, 1, which allowed for consistent compatibility with binary loss and activation functions such as binary cross entropy and sigmoid function during training. Subsequently, the main feature values also had to be normalized from their original range of -1, 0, 1 to 0, 0.5, 1 to preserve the relationships of feature values across all inputs. As the UCI_2015 dataset utilizes categorical features, such linear scaling was favored over other methods that are preferred for continuous distributions. The Mendeley_2020 dataset comprises of missing values that are represented by -1, which are harmful as they can disrupt the training of neural networks due to their introduction of incomplete feature vectors. To address this, a SimpleImputer from Scikit-Learn was utilized to incorporate a median imputation as it will make it robust to extreme outliers while preserving the central distribution of the features. Following this, normalization was carried out using the standardized z-score normalization method which rescales each feature using its mean and standard deviation. This normalization method improves the performance of a neural network by ensuring equal contribution from all features without any dominance, while also improving convergence stability and speed.

$$z = \frac{(x - \mu)}{\sigma} \quad (7)$$

After pre-processing the datasets, they were split into training and test subsets with an 80:20 splitting ratio. Since both datasets are heavily unbalanced, Random Over Sampling (ROS) was applied to the training sets of both datasets to address class imbalance [20] and ensure a more balanced distribution of samples for effective model learning. Additionally, stratified 5-Fold Cross Validation was employed for further improvements to robustness and generalization during the final performance evaluation.

As phishing detection is a binary classification problem, binary cross entropy loss function along with sigmoid activation function were utilized to obtain optimal classification probability. Rectified Linear Unit (ReLU) was utilized as the main activation function for the convolutional and dense layers along with the L2 regularizer as the kernel regularizer for every convolution operation. Adam optimizer was employed due to its adaptive learning rate capabilities along with a learning rate scheduling strategy to dynamically adjust the learning rate during training. This allows the model to learn efficiently and tune its parameters dynamically. 30 epochs was the standard for training all models in 3 different batch

sizes of 16, 32 and 64. Additionally, an early stopping technique was utilized to prevent the model from overfitting. Accuracy was identified as the primary metric for evaluation.

Due to the vast quantity of hyperparameters that can influence the framework's performance and structure, it would be inconvenient to test every configuration manually. A python framework called Optuna was employed to identify optimal model configurations in one go. Each of the 3 batch sizes underwent 30 trials, for both datasets bringing the total number of trials to 180, with each trial training a unique configuration for 30 epochs.

5.4 Results Analysis

This section will cover a comprehensive evaluation of the results obtained from the experiments conducted as part of this project. This analysis will include an ablation study followed by a comparison of performance across datasets and against baseline models. Lastly, the section will include a comparison of the hybrid framework with previous research.

5.4.1 Ablation Study. :

The purpose of this ablation study is to systematically evaluate the contribution of each architectural component of the proposed hybrid framework. By isolating and progressively combining the components, this study aims to understand each component's influence on the classification accuracy and generalization across datasets. Three model variants were implemented and evaluated under identical training configurations to analyze their effectiveness. Additionally, to evaluate the generalization capability, all model variants were tested on both datasets.

- **1D-CNN Only:** Comprises only of convolutional layers followed by pooling, batch normalization and dense layers.
- **BiGRU Only:** Relies exclusively on capturing sequential dependencies in both directions.
- **1D-CNN + BiGRU (Proposed Framework):** Hybrid configuration with convolutional outputs being passed onto BiGRU before classification, combining spatial and temporal representations.

Fig.7. showcases the performance increase on the UCI_2015 dataset from 96.11% (1D-CNN Only) to 97.92% with the full hybrid configuration. Similarly, the Mendeley_2020 dataset exhibits progressive improvement, culminating in 96.61% accuracy for the proposed model. Although the Mendeley_2020 dataset achieved a slightly lower accuracy compared to the UCI_2015 dataset, the difference of approximately 1.3% suggests moderate dataset sensitivity while maintaining strong generalization capability. The relatively small performance gap indicates that the hybrid framework effectively adapts to variations in feature distribution and data complexity.

Furthermore, to evaluate the architectural performance, the model variants were assessed using the empirical metrics across both datasets. The results in Table. 6. demonstrate a consistent and progressive improvement in classification accuracy as architectural components are incrementally integrated across both datasets.

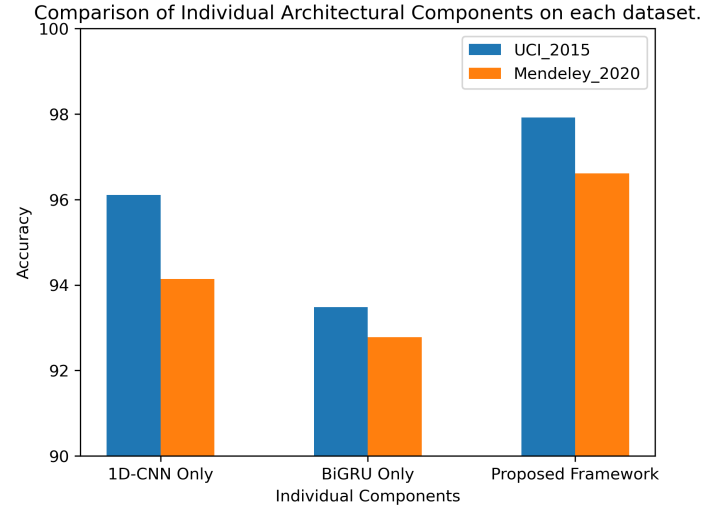


Fig. 7. Comparison of Individual Architectural Components on each dataset.

On the UCI dataset, the 1D-CNN baseline achieves strong performance with 96.11% accuracy. However, the BiGRU only configuration performs noticeably lower across all metrics, indicating that sequential modeling alone is insufficient without spatial feature extraction. The combination of 1D-CNN and BiGRU significantly enhances performance, improving accuracy, precision and recall beyond the standalone models. This confirms that convolutional spatial feature extraction and bidirectional temporal dependency modeling provide complementary representational strengths. The proposed hybrid framework achieves the highest performance across all metrics, reaching 97.92% accuracy and 97.64% F1-score on the UCI dataset. The simultaneous increase in precision (98.05%) and recall (97.24%) demonstrates improved balance between FP reduction and detection sensitivity.

Dataset	Model Variant	Accuracy	Precision	Recall	F1
UCI	1D-CNN	96.11%	95.06%	96.22%	95.63%
	BiGRU	93.48%	91.46%	94.08%	92.75%
	Proposed Hybrid Framework	97.92%	98.05%	97.24%	97.64%
Mendeley	1D-CNN	94.14%	94.91%	94.34%	94.62%
	BiGRU	92.78%	93.94%	92.63%	93.27%
	Proposed Hybrid Framework	96.61%	94.79%	95.43%	95.11%

Table 6. Performance comparison of Individual Architectural Components on each dataset.

A similar trend is observed on the Mendeley 2020 dataset. Although the peak accuracy (96.61%) is approximately 1.3% lower than that of the UCI dataset, the performance progression remains consistent. The hybrid model again achieves the highest accuracy, precision, recall, and F1-score, confirming that architectural improvements generalize effectively across datasets with different distributions and complexity levels.

The confusion matrix provides detailed insight into how the model distinguishes between phishing and legitimate websites for the UCI_2015 dataset. The results in Fig. 8. indicate that the model correctly identifies the vast majority of both phishing and legitimate instances. The relatively small number of false positives suggests that legitimate websites are rarely misclassified as phishing, which is important for maintaining user trust and minimizing unnecessary alerts. Similarly, the low number of false negatives demonstrates that the system is highly effective at detecting phishing instances, reducing the likelihood of malicious websites by-passing the detection system. A similar trend is observed for the Mendeley_2020 dataset in Fig. 9 which despite having a slightly more complex feature distribution, the hybrid framework managed to maintain a strong predictive capability. The relatively small difference in performance compared to the UCI_2015 dataset suggests the proposed hybrid framework generalizes well across datasets with varying characteristics.

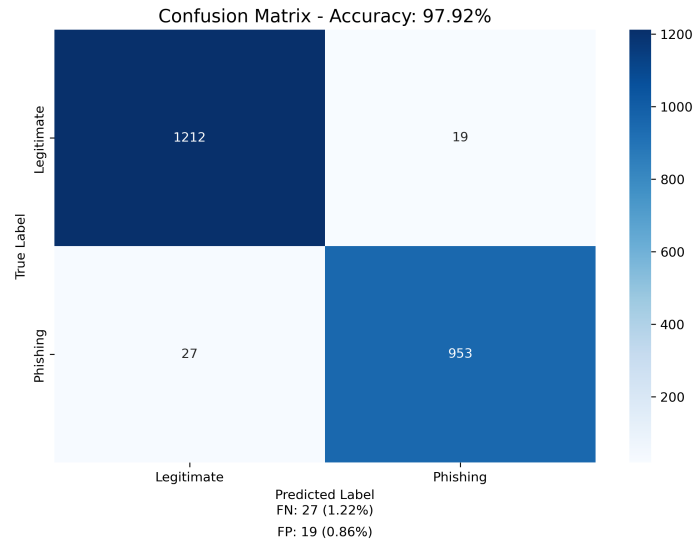


Fig. 8. Confusion Matrix for UCI_2015 dataset

Receiver Operating Characteristic (ROC) is a curve that plots the True Positive Rate (TPR) against the False Positive Rate (FPR) at different classification thresholds to further highlight the hybrid framework's classification capability. As shown in Fig. 10, the ROC curve closely approaches the top-left corner of the graph, indicating excellent discriminative ability between the two phishing and legitimate website classes. The resulting Area Under the Curve (AUC) value of 0.9947 demonstrates the model's ability to maintain a very high true positive rate while keeping the false positive rate minimal across different thresholds.

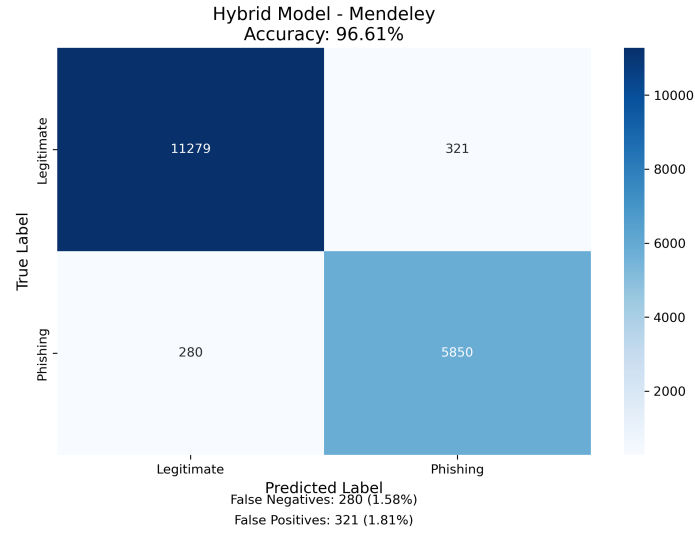


Fig. 9. Confusion Matrix for Mendeley_2020 dataset

This confirms the robustness of the hybrid framework in distinguishing phishing websites from legitimate websites. The ROC characteristics of the model on the Mendeley 2020 dataset in Fig. 11 demonstrate a similar pattern, although the AUC value is slightly lower compared to the UCI dataset, it still reflects strong discriminative capability. This indicates that the hybrid framework consistently maintains high true positive rates while limiting false positives across different dataset distributions.

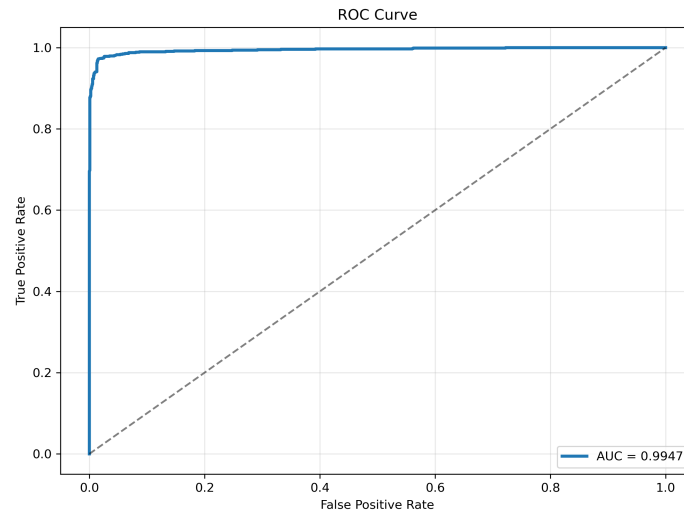


Fig. 10. ROC Curve for UCI_2015 dataset

Additionally, the Precision-Recall (PR) curve was analyzed to assess the model's performance in terms of precision and recall trade-offs. As illustrated in Fig. 12, the proposed

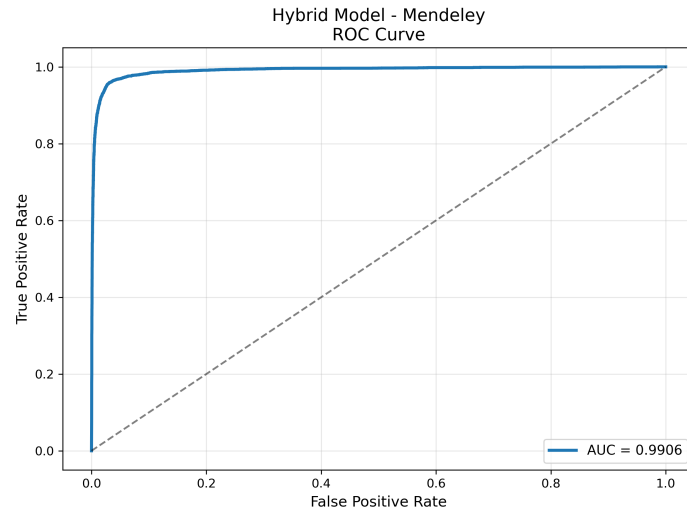


Fig. 11. ROC Curve for Mendeley_2020 dataset

model achieved a Precision-Recall AUC value of 0.9948, indicating an excellent balance between precision and recall across varying thresholds. The curve maintains precision values close to 1.0 over a wide range of recall levels, demonstrating that the model can identify phishing websites with very high confidence while maintaining minimal false detections. The slight decline in precision at extremely high recall values reflects the expected trade-off when attempting to capture nearly all phishing instances. When evaluated on the Mendeley_2020 dataset, the Precision-Recall curve, as seen in Fig. 13, exhibits comparable behavior, maintaining high precision values across a wide recall range. This suggests that the model is capable of reliably detecting phishing instances even when trained and evaluated on datasets with differing feature distributions and sizes.

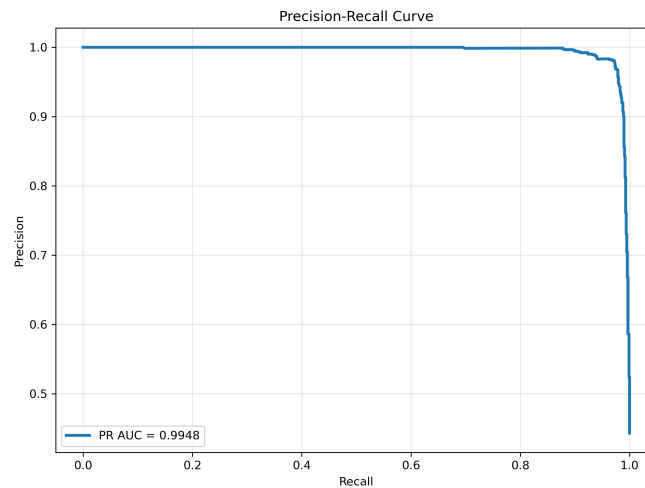


Fig. 12. PR Curve for UCI_2015 dataset

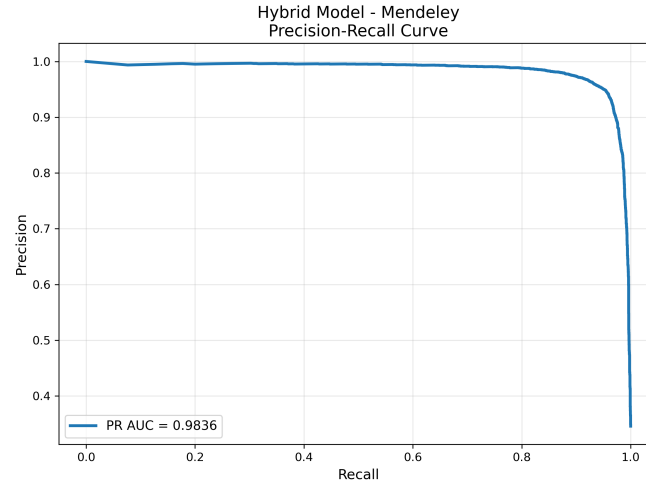


Fig. 13. PR Curve for Mendeley_2020 dataset

Furthermore, to evaluate the training behavior and generalization capability, the evolution of training and validation loss and accuracy was monitored across epochs, as shown in Fig. 14 & Fig. 15. The training loss exhibits a consistent decline throughout the training process, indicating that the model successfully learns discriminative feature representations. The validation loss also decreases rapidly during the early training stages before stabilizing, suggesting effective convergence. The validation accuracy quickly approaches the target threshold and stabilizes close to the final accuracy, demonstrating strong generalization capability. Most important is the relatively small gap between training and validation curves, indicating minimal overfitting. Training behavior on the Mendeley_2020 dataset follows a similar convergence pattern, with both training and validation metrics stabilizing after several epochs. While the final validation accuracy is slightly lower compared to the UCI_2015 dataset, the learning curves indicate stable convergence and minimal overfitting.

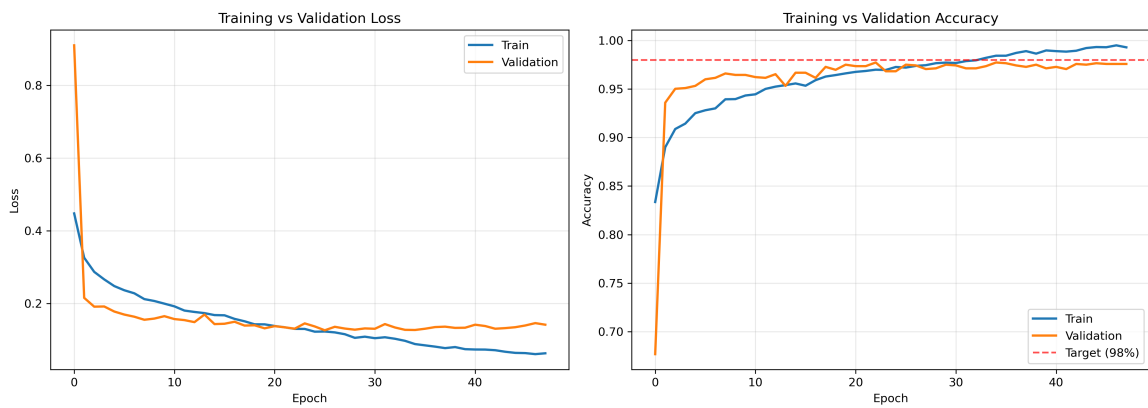


Fig. 14. Training vs Validation Loss & Accuracy on UCI_2015 Dataset.

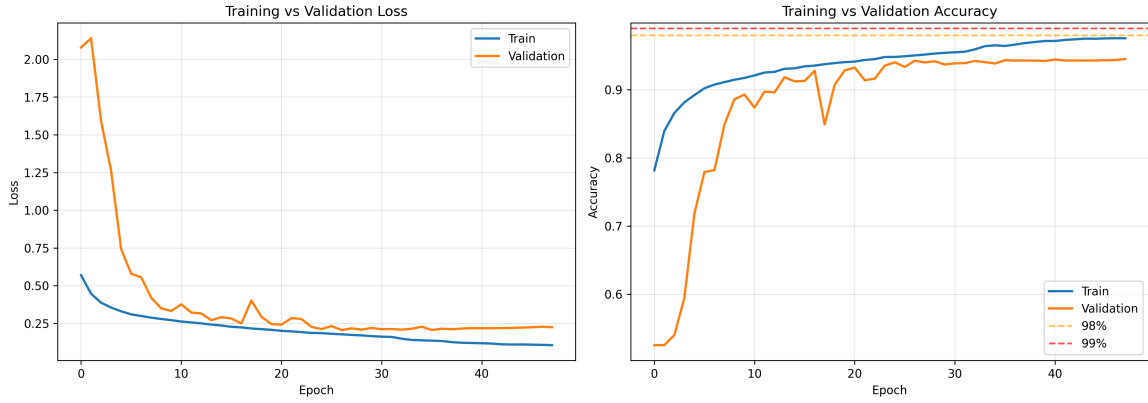


Fig. 15. Training vs Validation Loss & Accuracy on Mendeley_2020 Dataset.

To further evaluate the effectiveness of the proposed hybrid framework, several traditional Machine Learning models were implemented as baseline classifiers to provide a benchmark for comparison. The selected baseline models include Decision Trees, Naive Bayes, Support Vector Machine, Random Forest and Logistic Regression. These algorithms were chosen because they are commonly used in phishing detection research and provide a meaningful benchmark for evaluating deep learning approaches. All traditional models were trained and evaluated using the same dataset splits and feature representations used for the deep learning experiments to ensure a fair comparison. The performance of these models was assessed using the same four empirical metrics.

The results from Table. 7. indicate that traditional ensemble-based approaches such as Random Forest achieved the strongest performance among the traditional models, obtaining an accuracy of 94.21% and an F1-score of 93.33%. Decision Tree and Support Vector Machine models also demonstrated strong classification capability, achieving accuracies of 93.03% and 92.53%, respectively. Logistic Regression showed comparable performance with an accuracy of 92.76%. In contrast, the Naive Bayes classifier exhibited significantly lower overall accuracy (61.19%) despite achieving an extremely high recall of 99.89%. This behavior suggests that the model classified nearly all instances as phishing, resulting in a large number of false positives and consequently poor precision. Such imbalance highlights the limitations of probabilistic independence assumptions when applied to complex phishing datasets. Although Random Forest achieved the strongest results among the traditional classifiers, its performance remains lower than that of the proposed hybrid framework, which achieved an accuracy of 97.92% and superior overall classification metrics. This improvement demonstrates the ability of the hybrid architecture to capture both spatial feature patterns and sequential dependencies within the data more effectively than traditional learning algorithms.

A similar trend was observed for the Mendeley 2020 dataset as evident in Table.X. Decision Tree achieved the highest accuracy among the traditional models (92.82%), closely followed by Random Forest (91.85%) and Support Vector Machine (89.84%). Logistic Regression produced comparable performance with an accuracy of 89.44%. Naive Bayes once again

Dataset	Model	Accuracy	Precision	Recall	F1
UCI_2015	Decision Tree	93.03%	93.65%	90.40%	92.00%
	Naive Bayes	61.19%	53.32%	99.89%	69.53%
	SVM	92.53%	93.39%	89.48%	91.40%
	Random Forest	94.21%	95.31%	91.42%	93.33%
	Logistic Regression	92.76%	93.43%	90.00%	91.68%
	Proposed Framework	97.92%	98.05%	97.24%	97.64%
Mendeley_2020	Decision Tree	92.82%	92.06%	94.42%	93.22%
	Naive Bayes	58.27%	97.02%	20.78%	34.23%
	SVM	89.84%	90.42%	90.11%	90.26%
	Random Forest	91.85%	91.59%	92.95%	92.26%
	Logistic Regression	89.44%	90.42%	89.24%	89.83%
	Proposed Framework	96.61%	94.79%	95.43%	95.11%

Table 7. Comparison of performance of proposed framework against traditional ML models.

displayed unstable behavior, achieving a high precision (97.02%) but extremely low recall (20.78%), indicating that the model failed to correctly identify a large portion of phishing instances. This imbalance resulted in a low F1-score (34.23%), further confirming its unsuitability for this classification task. Compared to these baseline models, the proposed hybrid framework achieved a much higher classification accuracy of 96.61% on the Mendeley 2020 dataset while maintaining strong precision and recall values. This demonstrates that the hybrid deep learning architecture generalizes effectively across different datasets and consistently outperforms conventional machine learning approaches.

5.4.2 Comparison With Previous Works. :

This section shall provide a comparison of the proposed framework's performance with the existing phishing detection models reported in the previous studies in order to further evaluate the hybrid framework's effectiveness.

From the results in Table. 8. it can be summarized that when evaluated on the UCI_2015 dataset, the proposed hybrid framework achieved an accuracy of 97.92%, surpassing the high performance of the MLSELM approach (97.76%). This improvement suggests that the combination of convolutional feature extraction and temporal dependency modelling

Dataset	Model	Accuracy
UCI_2015 [24]	CART [29]	91%
	Ensemble Learning [37]	95.40%
	GA-based Gradient Boost [2]	97.13%
	EnLeM [42]	97.21%
	XGBoost [30]	97.45%
	MLSELM [20]	97.76%
	Proposed Hybrid Framework	97.92%
Mendeley_2020 [38]	FNN [25]	95.53%
	XGBoost [28]	96.34%
	Proposed Hybrid Framework	96.61%
	MLSELM [20]	96.79%
	GA-based XGBoost [2]	97.35%

Table 8. Comparison of performance of proposed framework against previous studies.

enables the framework to effectively capture complex relationships. The strong performance on the UCI dataset indicates that the hybrid architecture is capable of learning meaningful structural patterns within URL-based feature representations while maintaining competitive performance relative to other methods. However, when evaluated on the Mendeley_2020 dataset, the proposed model achieved a comparatively lower accuracy of 96.61%, which is higher than some previously reported approaches such as FNN (95.53%) and XGBoost (96.34%) but weaker than some other approaches such as MLSELM (96.79%) and GA-based XGBoost (97.35%). This difference suggests that the model exhibits some sensitivity to variations in dataset characteristics, such as feature distribution, dataset size, and the level of noise present within the data. The obtained performance still remains competitive and demonstrates stable results across two different datasets. It is also important to note that some studies reporting higher accuracies, such as Random Forest (99.06%), RNN-GRU (99.18%), and CNN (99.20%), were evaluated on entirely different datasets. Differences in dataset composition and feature extraction methods can significantly influence classification outcomes. Overall, while the proposed framework achieves its strongest results on the UCI_2015 dataset, it still demonstrates reliable performance and reasonable generalization capability when applied to other phishing detection datasets.

6 Conclusions

This dissertation proposes a hybrid deep learning framework to address an existing research gap in phishing websites detection. The proposed hybrid framework employs and integrates a 1-Dimensional Convolutional Neural Network (1D-CNN) with a Bidirectional Gated Recurrent Unit (BiGRU), capable of effectively identifying malicious URLs using deep learning techniques.

The research began with an extensive literature review (refer to Section 2), examining existing detection methodologies and datasets and highlighting their efficiency in detecting phishing websites, which highlighted the scarcity of models combining temporal dependencies with lexical features extraction. As such, an architecture for a hybrid deep learning framework (refer to Section 3) was conceived to address this research gap. This architecture sees the employment of a 1D-CNN to extract the local lexical features from the URLs of phishing websites, while the employment of a BiGRU component captures the temporal dependencies of these lexical URL tokens for better understanding the sequential relations between them. Together, these models form the end-to-end hybrid framework capable of detecting and identifying phishing websites.

The proposed model was evaluated using two state-of-the-art phishing datasets with experimental evaluation demonstrating strong performance across both datasets, achieving an accuracy of 97.78% on the UCI_2015 dataset and 95.20% on the Mendeley_2020 dataset. The results of the ablation study further confirmed the contribution of each architectural component, showcasing improved classification performance through progressive integration. Additionally, comparisons with traditional machine learning models and previously proposed approaches indicated that the hybrid framework achieves competitive performance while maintaining stable results across multiple datasets. These findings suggest that combining convolutional and recurrent architectures can effectively improve performance for phishing detection.

6.1 Limitations

Despite the promising results demonstrated by the proposed hybrid framework in this study, several limitations need to be acknowledged. Firstly, the datasets used in this research, although state-of-the-art, may not fully capture the diversity and continuously evolving nature of real-world phishing attacks. Phishing techniques frequently change as attackers develop new strategies, and static datasets may not always generalize well to unseen phishing patterns.

Second, the performance of the proposed hybrid framework varied across datasets, with the model achieving higher accuracy on the UCI dataset compared to the Mendeley 2020 dataset. This variation suggests that the model may exhibit some sensitivity to differences in dataset characteristics, needing further improvement to ensure consistent performance.

Lastly, the utilization of 1D-CNN and BiGRU deep learning frameworks enforce high computational complexity and costs, despite the relatively high accuracy. This will provide a challenge when attempting to deploy the model in real-time detection systems. Simpler traditional ML models fare better in this regard.

6.2 Scope for Future Work

Based on the limitations identified and the results obtained in this study, several directions for future research can be explored. One important area for improvement is the use of larger and more diverse phishing datasets that include URLs collected from multiple sources and real-world environments. Expanding the dataset diversity would allow the model to learn more generalized patterns and improve its robustness against evolving phishing techniques.

Another potential direction involves exploring alternative deep learning architectures to further enhance phishing detection performance such as transformer-based architectures or attention mechanisms could be incorporated to capture more complex feature relationships within URL structures. Furthermore, future research could investigate methods for reducing computational complexity while maintaining high performance.

Lastly, future studies may also explore the integration of additional feature types, such as webpage content, domain information, or behavioral features (emails), to complement URL-based detection for further enhancing performance and robustness.

References

- [1] S.R. Abdul Samad, S. Balasubramanian, A.S. Al-Kaabi, B. Sharma, S. Chowdhury, A. Mehbodniya, J.L. Webber, and A. Bostani. 2023. Analysis of the performance impact of fine-tuned machine learning model for phishing URL detection. *Electronics* 12(7) (2023), p.1642.
- [2] M. Al-Sarem, F. Saeed, Z.G. Al-Mekhlafi, B.A. Mohammed, T. Al-Hadhrani, M.T. Alshammari, A. Alreshidi, and T.S. Alshammari. 2021. An optimized stacking ensemble model for phishing websites detection. *Electronics* 10(11) (2021), p.1285.
- [3] M.M. Ali and N.F. Mohd Zaharon. 2024. Phishing—A cyber fraud: The types, implications and governance. *International Journal of Educational Reform* 33(1) (2024), pp.101–121.
- [4] A. Almomani, M. Alauthman, M.T. Shatnawi, M. Alweshah, A. Alrosan, W. Alomoush, and B.B. Gupta. 2022. Phishing website detection with semantic features based on machine learning classifiers: a comparative study. *International Journal on Semantic Web and Information Systems (IJSWIS)* 18(1) (2022), pp.1–24.
- [5] N.F. Almujaheed, M.A. Haq, and M. Alshehri. 2024. Comparative evaluation of machine learning algorithms for phishing site detection. *PeerJ Computer Science* 10 (2024), p.2131.
- [6] S. Alnemari and M. Alshammari. 2023. Detecting phishing domains using machine learning. *Applied Sciences* 13(8) (2023), p.4649.
- [7] Z. Alshingiti, R. Alaqel, J. Al-Muhtadi, Q.E.U. Haq, K. Saleem, and M.H. Faheem. 2023. A deep learning-based phishing detection system using CNN, LSTM, and LSTM-CNN. *Electronics* 12(1) (2023), p.232.
- [8] Simard P. Bengio, Y. and P. Frasconi. 1994. Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE transactions on neural networks* 5(2) (1994), pp.157–166.
- [9] Google Safe Browsing. n.d. Google Safe Browsing: keeping over five billion devices safer [online]. Available at: <https://safebrowsing.google.com/>. (Accessed: 2025-10-17).
- [10] Y. Cao, W. Han, and Y. Le. 2008. Anti-phishing based on automated individual white-list. In *Proceedings of the 4th ACM workshop on Digital identity management*. pp.51–60.
- [11] C.M.R. da Silva, E.L. Feitosa, and V.C. Garcia. 2020. Heuristic-based strategy for Phishing prediction: A survey of URL-based approach. *Computers Security* 88 (2020), p.101613.
- [12] A. Fedeale, M. Tonin, and M. Valerio. 2024. Phishing attacks: An analysis of the victims' characteristics based on administrative data. *Economics Letters* 237 (2024), p.111663.
- [13] Canadian Institute for Cybersecurity. 2016. URL dataset (ISCX-URL2016) [dataset]. Available at: <https://www.unb.ca/cic/datasets/url-2016.html>. (Accessed: 2025-10-18).
- [14] Anti-Phishing Working Group. 2025. Phishing Activity Trends Report – 2nd Quarter 2025. Available at: https://docs.apwg.org/reports/apwg_trends_report_q2_2025.pdf. (Accessed: 2025-10-16).
- [15] M. Guo, G. Wang, H. Hata, and M.A. Babar. 2021. Revenue maximizing markets for zero-day exploits. *Autonomous Agents and Multi-Agent Systems* 35(2) (2021), p.36.
- [16] A.K. Jain and B.B. Gupta. 2016. A novel approach to protect against phishing attacks at client side using auto-updated white-list. *EURASIP Journal on Information Security* 2016(1) (2016), p.9.
- [17] A.K. Jain and B.B. Gupta. 2017. Phishing detection: analysis of visual similarity based approaches. *Security and Communication Networks* 2017(1) (2017), p.5421046.
- [18] J. James, L. Sandhya, and C. Thomas. 2013. Detection of phishing URLs using machine learning techniques. In *2013 international conference on control communication and computing (ICCC)*. pp.304–309.
- [19] J.B. Joolee, M.A. Uddin, and S. Jeon. 2022. Deep multi-model fusion network based real object tactile understanding from haptic data. *Applied Intelligence* 52(14) (2022), pp.16605–16620.
- [20] L.R. Kalabarige, R.S. Rao, A. Abraham, and L.A. Gabralla. 2022. Multilayer stacked ensemble learning model to detect phishing websites. *IEEE Access* 10 (2022), pp.79543–79552.
- [21] Y Kim. 2014. Convolutional Neural Networks for Sentence Classification. In *2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. pp.1746–1751.

- [22] Avci O. Abdeljaber O. Ince T. Gabbouj M. Kiranyaz, S. and D.J. Inman. 2021. 1D Convolutional Neural Networks and Applications: A Survey. *Mechanical Systems and Signal Processing* 151 (2021), p.107398.
- [23] Y. LeCun, Y. Bengio, and G. Hinton. 2015. Deep learning. *Nature* 521(7553) (2015), pp.436–444.
- [24] R. Mohammad and L. McCluskey. 2015. Phishing Websites. UCI Machine Learning Repository. Available at: <https://doi.org/10.24432/C51W2X>. (Accessed: 2025-10-19).
- [25] Muniyal B. Nayak, G.S. and M.C. Belavagi. 2025. Enhancing Phishing Detection: A Machine Learning Approach With Feature Selection and Deep Learning Models. *IEEE Access* (2025).
- [26] PhishTank. n.d. PhishTank: a free community site for submitting and verifying suspected phishing websites and emails [online]. Available at: <https://www.phishtank.com/>. (Accessed: 2025-10-17).
- [27] Kumar M. Kompella R.R. Prakash, P. and M. Gupta. 2010. Phishnet: predictive blacklisting to detect phishing attacks. In *2010 Proceedings IEEE INFOCOM*. pp.1–5.
- [28] Ghazi W. Prameswari, N. and F.A. Rafrastara. 2026. Systematic XGBoost Pipeline for Phishing Website Detection: Hyperparameter Tuning Approach with Nested Cross-Validation. *Jurnal Ilmiah Bidang Teknologi Informasi dan Komunikasi* 11(1) (2026), pp.8–110.
- [29] Haeruddin H. Prasetyo, S. E. and A. 2024. Phishing Website Detection Using the Decision Tree Algorithm Method. *Jurnal E-Komtek (Elektro-Komputer-Teknik)* 8(2) (2024), pp.454–465. Available at: <https://doi.org/10.37339/e-komtek.v8i2.2167>.
- [30] Alfisyahrin A.R. Gambetta W. Karimah S.A. Prayogo, R.D. and H. Nambo. 2024. An Explainable Machine Learning-Based Phishing Website Detection using Gradient Boosting. In *2024 International Conference on Information Technology Research and Innovation (ICITRI)*. pp.76–81.
- [31] O.K. Sahingoz, E. Buber, O. Demir, and B. Diri. 2019. Machine learning based phishing detection from URLs. *Expert Systems with Applications* 117 (2019), pp.345–357.
- [32] J.H. Setu, N. Halder, A. Islam, and M.A. Amin. 2025. RSTHFS: A Rough Set Theory-Based Hybrid Feature Selection Method for Phishing Website Classification. *IEEE Access* 13 (2025), pp.68820–68830.
- [33] D. She and M. Jia. 2021. A BiGRU method for remaining useful life prediction of machinery. *Measurement* 167 (2021), p.108277.
- [34] C.L. Tan. 2018. Phishing dataset for machine learning: Feature evaluation. Mendeley Data, 1(8), V1. Available at: <https://doi.org/10.17632/h3cgnj8hft.1>. (Accessed: 2025-10-19).
- [35] L. Tang and Q.H. Mahmoud. 2021. A deep learning-based framework for phishing website detection. *IEEE Access* 10 (2021), pp.1509–1521.
- [36] I. Tyagi, J. Shad, S. Sharma, S. Gaur, and G. Kaur. 2018. A novel machine learning approach to detect phishing websites. In *2018 5th International conference on signal processing and integrated networks (SPIN)*. pp.425–430.
- [37] Jasmi S.K.B. Abdullah A. Jhanjhi N.Z. Ubing, A.A. and M. Supramaniam. 2019. Phishing Website Detection: An Improved Accuracy through Feature Selection and Ensemble Learning. *International Journal of Advanced Computer Science and Applications* 10(1) (2019).
- [38] G. Vrbančić. 2020. Phishing Websites Dataset. Mendeley Data, V1. Available at: <https://doi.org/10.17632/72ptz43s9v.1>. (Accessed: 2025-10-19).
- [39] M. Wang, L. Song, L. Li, Y. Zhu, and J. Li. 2024. Phishing webpage detection based on global and local visual similarity. *Expert Systems with Applications* 252 (2024), p.124120.
- [40] Y. Wei and Y. Sekiya. 2022. Sufficiency of ensemble machine learning methods for phishing websites detection. *IEEE Access* 10 (2022), pp.124103–124113.
- [41] G. Xiang, J. Hong, C.P. Rose, and L. Cranor. 2011. Cantina+ a feature-rich machine learning framework for detecting phishing web sites. *ACM Transactions on Information and System Security (TISSEC)* 14(2) (2011), pp.1–28.
- [42] Refat M.A.R. Singh B.C. Alom Z. Aung Z. Yeasmin, M.N. and M. Azim. 2025. EnLeM: ensemble learning-based model to detect phishing websites. *EURASIP Journal on Information Security* 2026(1) (2025), p1.
- [43] Y. Zhang, J.I. Hong, and L.F. Cranor. 2007. Cantina: a content-based approach to detecting phishing web sites. In *Proceedings of the 16th international conference on World Wide Web*. pp.639–648.

- [44] R. Zieni, L. Massari, and M.C. Calzarossa. 2023. Phishing or not phishing? A survey on the detection of phishing websites. *IEEE Access* 11 (2023), pp.18499–18519.

A Project Management

In order to ensure the research is consistent with the objectives and is completed on time without being rushed, proper planning and organization is required from the start until the end. This section outlines the scope and planning of the projects along with the placement of milestones to ensure progress is executed smoothly. Moreover, the section shall also contain an analysis of possible risks that can be encountered during the project, and ways to mitigate them.

A.1 Project Scope & Deliverables

This research aims to address the current limitation of phishing websites detection systems, specifically the lack of existing systems that combine lexical feature extraction from phishing URLs with temporal sequential dependencies to understand patterns and relations. To overcome this gap, this project proposes an end-to-end hybrid deep learning framework, integrating a 1-Dimensional Convolutional Neural Network (1D-CNN) for extracting lexical features, with a Bidirectional Gated Recurrent Unit (BiGRU) for capturing sequential dependencies.

The project scope can be divided into the following key tasks for its development:

- (1) Preprocessing publicly available phishing datasets.
- (2) Normalizing data for better training compatibility.
- (3) Implementing the 1D-CNN to extract lexical features.
- (4) Integrating a BiGRU model to extract temporal features.
- (5) Combining both models to form the hybrid framework.
- (6) Evaluating the performance of the framework using empirical metrics.
- (7) Comparing the performance of the framework against existing ML based systems.
- (8) Documenting possible risks that can be encountered and providing mitigation strategies.

Project Deliverables:

The project is divided into 5 deliverables, each of which has to be submitted before their specific deadlines. It is imperative for all tasks pertaining to each deliverable to be completed in an organized and timely fashion for smooth delivery. The deliverables are described below.

A.1.1 Project Proposal and Research Document (Semester 1). :

The first deliverable is the research document establishing the foundation of the project by highlighting a brief overview of the study and the preparation done for it. It shall state the project's aim and objectives concisely along with the background behind the motivation of the project. A comprehensive literature review of existing studies and research in the field of phishing detection systems shall be conducted. The methodology behind the project's proposed framework shall be established, along with the datasets and empirical metrics on which it will be evaluated. This report will also present a work plan with milestones,

possible risks associated with the project and provide strategies to mitigate them, as well as address any relevant professional, legal, ethical and social issues.

A.1.2 5-Minute Feasibility Video (Semester 1). :

The second deliverable is a short 5 minute video and presentation explaining the work done behind the overview and preparation for this project as well as an explanation of the feasibility of the proposed 1D-CNN-BiGRU hybrid framework's completion within planned timeframes.

A.1.3 Dissertation Report (Semester 2). :

The third deliverable is the final comprehensive dissertation, which documents the complete lifecycle of the project's implementation including detailed descriptions of the methodology, systems and algorithms. It shall also include the detailed evaluation process, along with an in-depth analysis of the performance evaluated against other models to establish its standing and contributions.

A.1.4 Project Management Repository (Semester 2). :

The fourth deliverable is a project management repository containing all project related materials including: relevant research papers, datasets, diagrams and figures, logbooks, documentations and source code files. The repository shall be maintained and updated throughout the development lifecycle in order to provide traceability as well as act as evidence of work.

A.1.5 Q&A Session (Semester 2). :

The final deliverable is a Q&A session between the author of this dissertation and the supervisor with an additional marker. It is here that the author's knowledge and capabilities on phishing detection systems with regards to the project's design and framework implementation will be tested.

A.2 Project Plan & Gantt Chart

The project plan serves as a structured timeline depicting the execution of every task from the beginning of the project's preliminary research, through to the end of every necessary deliverable. It highlights the important tasks, milestones and deadlines that need to be met for smooth completion of the project. The plan is divided into two semesters, each corresponding to the two main deliverables required. The Gantt charts shown in Figures 16-20 depict the planning and listing of tasks required for completion of each deliverable for each semester in time phased manner.

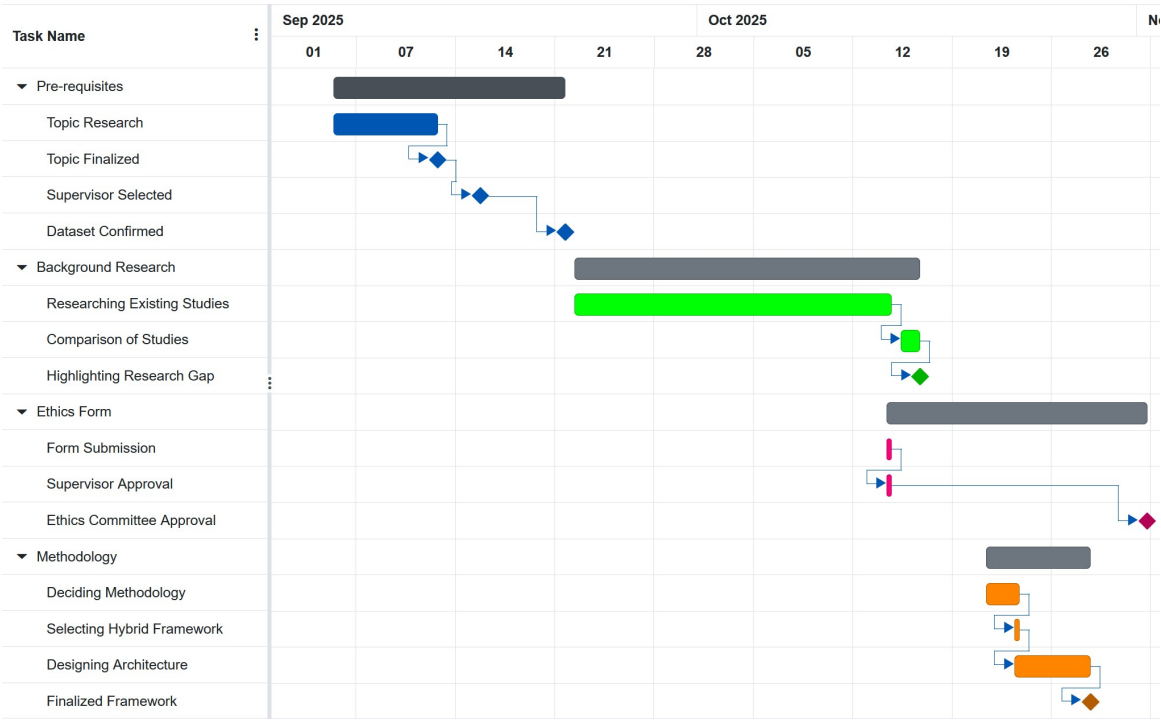


Fig. 16. Project Timeline for Semester 1 - Part 1

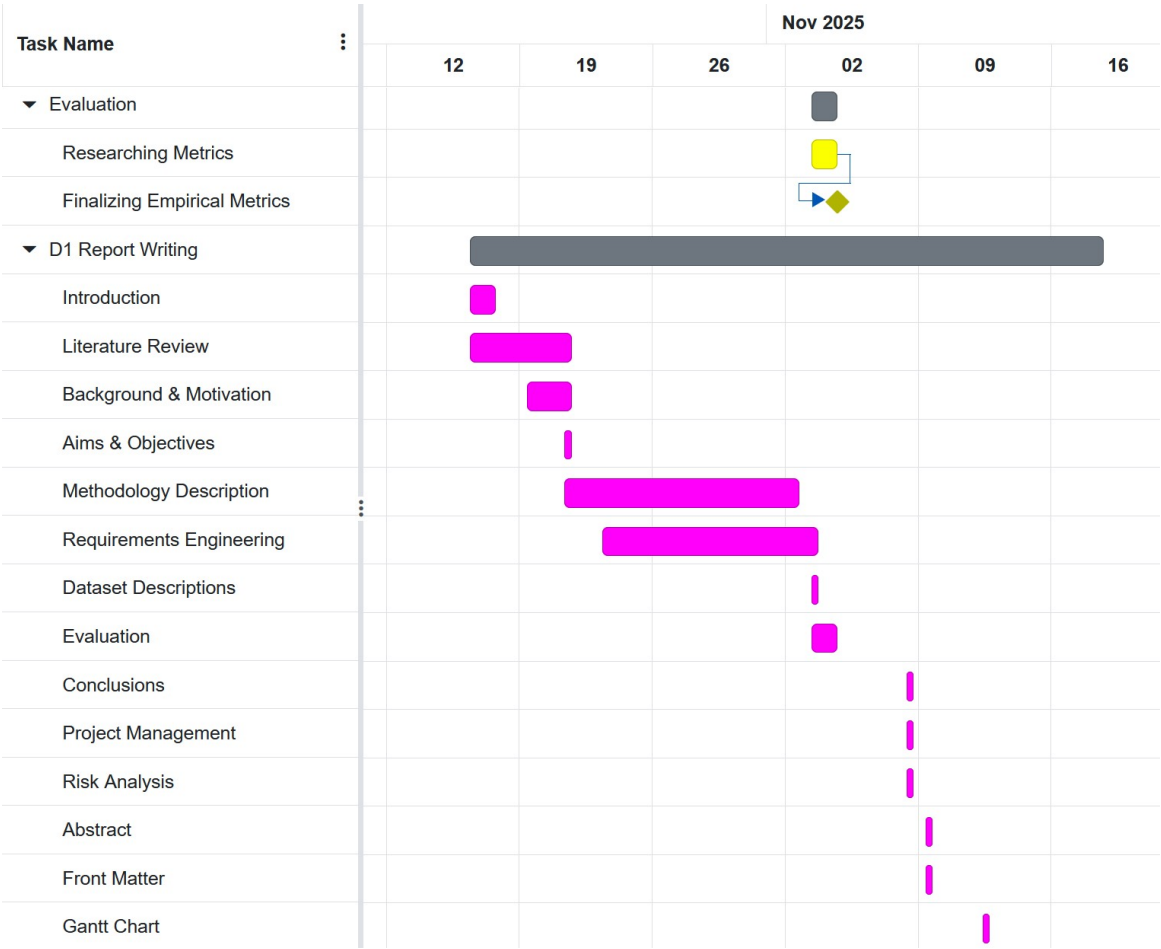


Fig. 17. Project Timeline for Semester 1 - Part 2



Fig. 18. Project Timeline for Semester 1 - Part 3

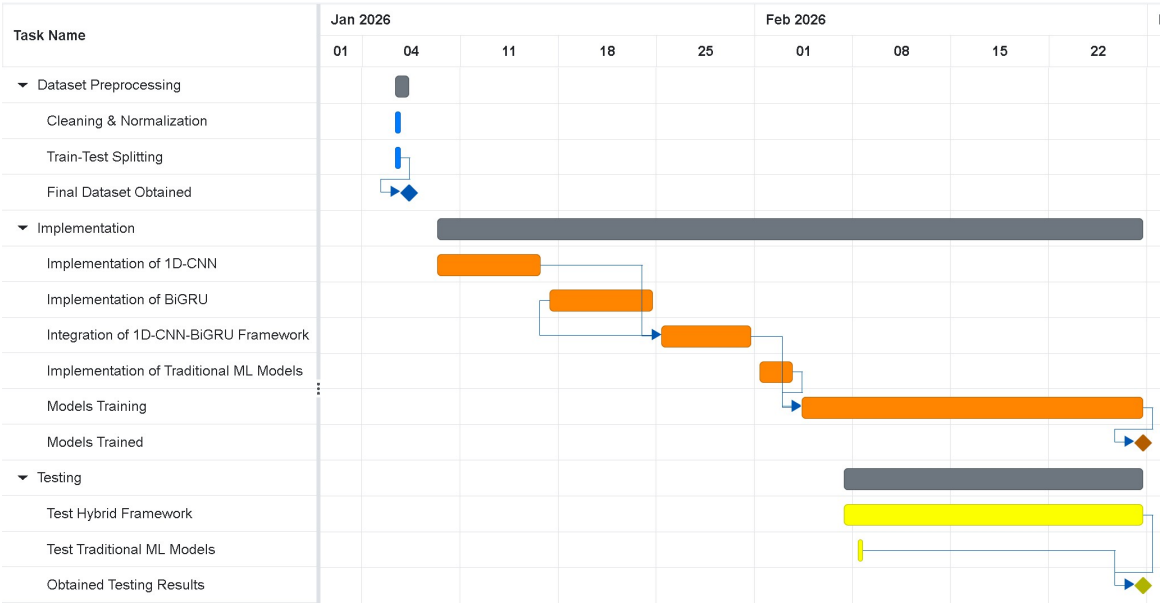


Fig. 19. Project Timeline for Semester 2 - Part 1

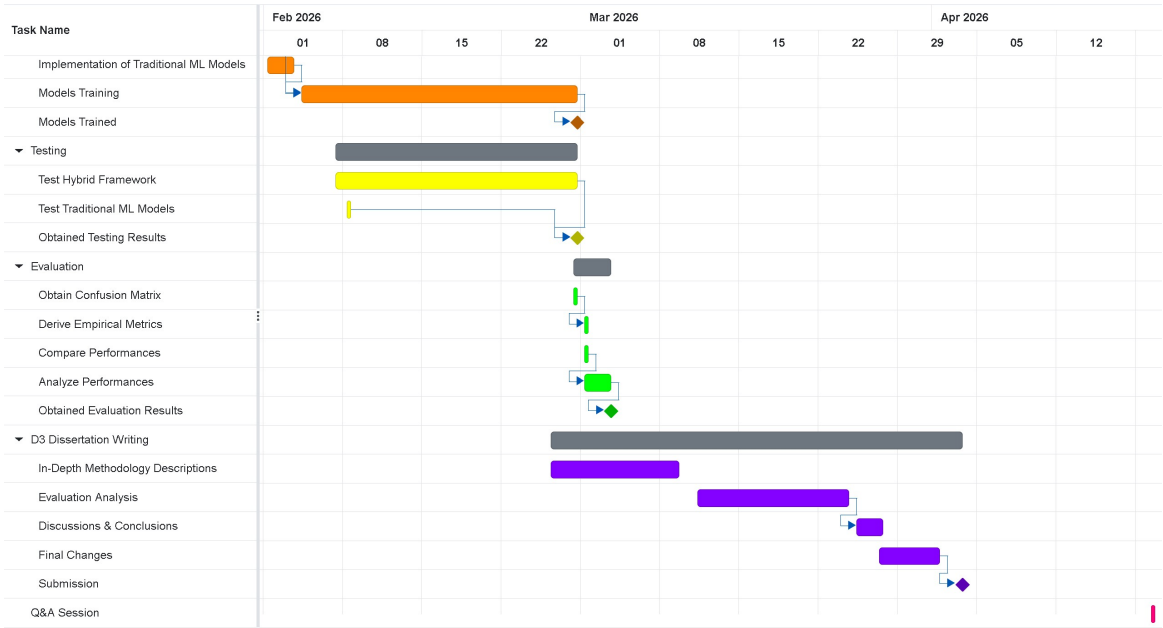


Fig. 20. Project Timeline for Semester 2 - Part 2

A.3 Risk Analysis

This section shall go over the possible risks that can be encountered during the development of the project. It is imperative that all possible risks are conceptualized as part of risk analysis so as to ensure that the project proceeds on schedule without any unforeseen complications. Table. 9 outlines the most probable risks, their likelihood and impact, as well as a possible strategy to mitigate or eliminate it. The likelihood can be classified as Very Likely, Likely, Possible and Unlikely. Impact can be classified as Very High, High, Medium and Low.

Risk	Likelihood	Impact	Mitigation Strategy
Low performance of hybrid framework.	Likely	High	Further hyper-parameter tuning, ensuring model is properly coded
Hybrid framework does not outperform other ML models.	Very Likely	High	Extensive hyper-parameter tuning.
Overfitting During Training	Likely	Medium	Implement cross validation, dropout or early stopping.
Inconsistent Evaluation Results.	Possible	High	Conduct multiple test runs and ensure consistent implementation of empirical metrics.
Data Corruption / Loss of Progress	Unlikely	Very High	Make regular copies and backups.
System Failure	Unlikely	Very High	Make backup saves to cloud storage.
Insufficient Hardware Capabilities & Resources	Unlikely	High	Utilize cloud-based environments such as Google Colab.
Time Constraints	Likely	Medium	Adhere to the planned timeline in the Gantt charts.
Bugs in developed code	Likely	Medium	Debugging and testing every feature in increments.

Table 9. Risk Analysis Table.

B Professional, Legal, Ethical and Social Issues

This project aligns with the standards and principles set by the institution and certified computing bodies to ensure responsible academic integrity and provide benefit to society.

B.1 Professional Issues

This project upholds strong standards in the procedures for collecting, preprocessing, evaluating and reporting on appropriate data. All systems, frameworks and models utilized follow current modern methodologies in machine learning and cybersecurity. This project relies only on employing well established open source tools such as Python, Scikit-Learn, TensorFlow, Keras, Pandas, NumPy, Matplotlib and any other library that may be relevant to detecting phishing websites, which are referenced within their terms of use. Furthermore, the professional procedures for ensuring and maintaining integrity and competence through development and documentation are consistent with the standards set by the British Computing Society (BCS).

B.2 Legal Issues

This project complies with relevant data protection laws, including regulations set by the United Arab Emirates and General Data Protection Regulation (GDPR) for the handling of datasets. The phishing datasets employed in this project consist of publicly available technical URL features, none of which contain any user-specific personal information. These datasets are utilized solely for the purpose of academic research under institutional supervision. Any performance data along with its evaluation, is stored locally and will not be accessed by anyone other than the author and project supervisor.

B.3 Ethical Issues

This project does not require nor does it utilize any form of human participation for its purpose. As such, no sensitive personal information pertaining to specific individuals is stored or processed. The datasets employed contain only technical URL features of publicly available phishing and legitimate websites, with no information that can trace back to any individual. The evaluation of the proposed framework's performance shall be fairly conducted and validated using empirical metrics only. This project respectfully adheres to the data management requirements set by Heriot-Watt University's ethics committee.

B.4 Social Issues

The purpose of this project is to contribute positively to the societal efforts against phishing detection by proposing a hybrid deep learning framework, capable of detecting and classifying malicious phishing websites with an increased accuracy and precision. The state-of-the-art datasets are widely used and recognized, containing only technical URL features and no human data. The project and its evaluation are purely technical without introducing or pertaining to any other broader sensitive social issue. Any correlation found is purely coincidental and unintentional.